

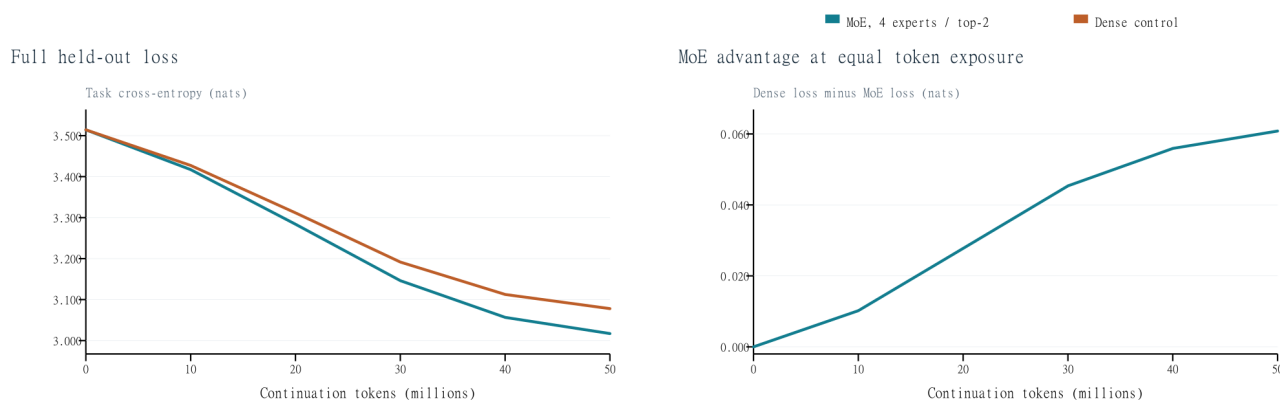
Dense-to-MoE upcycling

Successful conversion and continued learning. A 20.17M-parameter dense language model was converted into a 54.69M-parameter sparse MoE. After 50M continuation targets, full held-out loss reached **3.017127**, compared with **3.077944** for the token-matched dense control.

Stage	Dense loss	MoE loss
Before dense pretraining	9.118244	-
After 50M dense pretraining	3.514192	-
Immediately after conversion	3.514191	3.514189
After 10M continuation	3.427297	3.417105
After 50M continuation	3.077944	3.017127

More training: does the MoE advantage persist?

Paired continuation on the same 50M reused targets | full held-out evaluations | one training seed



Learning-rate schedule restarted at 10M for both arms. Equal tokens do not mean equal computation.

All loss values use 5,049,456 held-out supervised targets. MoE perplexity falls from 33.59 at conversion to 20.43. Loss excludes the auxiliary balancing penalty.

Assignment interpretation. The lesson asks to train a "Linear model," convert it to an MoE and reduce loss (p. 52). This work interprets that as the dense Transformer used in the upcycling lesson; the trained network includes nonlinear GELU feed-forward layers.

The architecture, exactly

Component	Value
Transformer / MoE layers	9 / 9
Hidden width / vocabulary / context	384 / 8,192 / 512
Query heads / KV heads / dimension	6 / 6 / 64; full multi-head attention
Routed experts per layer / total	4 / 36
Selected per token per layer / model	2 / 18 expert applications
Shared experts / expert parallelism	0 / no; single GPU
Each expert	384 -> 1,664 -> 384; GELU; two bias-free matrices
Each expert parameter count	1,277,952
Router per layer	384 -> 4; 1,536 parameters
All routers	13,824 parameters
Total MoE parameters	54,685,440
Active / inactive parameters per token	31,682,304 / 23,003,136
Expert bank / common parameters	46,006,272 / 8,679,168
Dense parameters	20,166,912

One block

Input -> LayerNorm -> causal attention -> residual addition -> LayerNorm -> top-2 MoE -> residual addition. Repeat for all nine layers; finish with LayerNorm and the tied vocabulary projection. Token and learned position embeddings both have width 384.

The router applies a FP32 softmax over four scores, selects the top two, and renormalizes their weights to sum to one. Tokens are gathered for each selected expert; weighted outputs are combined using sparse dispatch. There is no capacity cap or token dropping.

Why conversion preserves the starting function

Each dense FFN is copied four times. Initially all expert functions are identical. Since selected weights sum to one, any selected pair gives the original FFN output. The measured FP32 maximum logit difference is 0.000020981, below the 0.0002 tolerance.

Active does not mean the rest disappears

For one token, two experts per layer are skipped. Their parameters remain in memory, and other tokens may select them. All weights are trainable. The active-parameter convention counts the full tied embedding table and is not a FLOP measurement. Across training, all 36 expert weight sets changed. No expert is assigned a predefined topic or language.

Challenge the findings

Final-weight routing policy	Held-out loss	What it tests
Learned top-2 + learned weights	3.017127	Normal model
Learned top-2 + equal weights	3.103750	Sensitivity to mixture weights
Random top-2 + equal weights	3.465751 mean	Sensitivity to expert choices

Random routing uses inference seeds 17, 29 and 43. These are fixed-weight interventions, not independent training runs or models trained with random routing. Every policy still executes two experts per token.

How expert use changed with more training

Each cell is its share of routed assignments within a layer. Uniform use would be 25%.

After 10M continuation tokens

	E0	E1	E2	E3
Layer 1	16.9%	49.6%	30.2%	3.3%
Layer 2	33.6%	26.6%	20.6%	19.2%
Layer 3	19.6%	32.0%	27.2%	21.2%
Layer 4	31.6%	25.7%	27.1%	15.6%
Layer 5	26.1%	25.8%	24.1%	24.0%
Layer 6	24.1%	19.9%	28.8%	27.2%
Layer 7	25.1%	21.8%	25.6%	27.6%
Layer 8	22.1%	26.5%	26.8%	24.6%
Layer 9	27.7%	27.3%	20.8%	24.2%

Dead experts: 0 / 36

After 50M continuation tokens

	E0	E1	E2	E3
Layer 1	0.9%	49.9%	48.6%	0.6%
Layer 2	36.1%	31.6%	15.4%	16.9%
Layer 3	16.2%	40.7%	13.6%	29.5%
Layer 4	37.0%	20.6%	33.5%	8.9%
Layer 5	24.6%	28.1%	26.4%	20.9%
Layer 6	25.1%	18.6%	28.6%	27.8%
Layer 7	26.8%	20.4%	25.5%	27.3%
Layer 8	22.3%	28.1%	25.5%	24.1%
Layer 9	25.2%	28.2%	20.8%	25.8%

Dead experts: 0 / 36

Balance is incomplete. All 36 experts are selected somewhere in full validation, but layer 1's two busiest experts take 98.49% of assignments, versus 79.83% after 10M continuation targets. No zero-use expert is not the same as uniform utilization.

The comparison matches data exposure, not compute. MoE has more active parameters and higher elapsed time. Its 0.060817-nat advantage does not establish a compute-matched architectural advantage.

Loss is not fluency. Both final models produced repetitive or incoherent continuations on all six fixed greedy prompts. Complete outputs are included in evidence/stage2/GENERATIONS.md.

Single-seed evidence. This experiment uses one training seed per architecture, and the validation set was inspected during development. It demonstrates continued learning here; it does not establish statistical superiority, semantic expert specialization, or general language competence.

Training and reproducibility

Setting	Recorded value
GPU / software	RTX 3070 Laptop 8 GiB; Python 3.12.5; PyTorch 2.5.1+cu121
Data	50M training targets; same targets reused for continuation; 5,049,456 held-out targets
Batch / precision	32 sequences, continuation microbatch 16; BF16 matrix operations; FP32 weights, router and AdamW
Optimizer	AdamW (0.9, 0.95), epsilon 1e-8; matrix decay 0.1; clipping 1.0
Dense pretraining	3,277 updates, peak LR 6e-4, minimum 6e-5
First 10M continuation	656 updates, fresh optimizer; peak LR 1e-4, minimum 1e-5
Remaining 40M continuation	2,622 updates, preserve optimizer; peak LR 2e-4, minimum 2e-5
Auxiliary balance loss	Coefficient 0.001; valid tokens per microbatch; average across layers

40M extension	Elapsed	Targets/s	Peak allocated
Dense	11.59 min	57,520	2.19 GiB
MoE	18.77 min	35,522	3.99 GiB

Timing includes periodic validation and checkpoint saving, excluding setup, separate diagnostics, interruption downtime and discarded uncommitted work. The MoE resumed a verified update-1,500 optimizer checkpoint. The 10M/40M boundary masks are disjoint and exhaustive.

What is included

The GitHub package contains a detailed README, an executed local-review notebook, original dense and MoE code/results, architecture and critical-study guides, charts, the exact packed data and tokenizer, three final checkpoints, and portable setup instructions. Dataset arrays and checkpoints use Git LFS. Intermediate optimizer checkpoints are regenerated when retraining.

Start with **python run.py verify**, **python run.py tests**, and **python run.py check**. To reproduce the full MoE holdout score: **python run.py evaluate --kind moe** on a BF16-capable CUDA GPU. See SETUP.md for complete pretraining and continuation commands.

Packaging checks

The final MoE was loaded from the packaged files and evaluated on all 5,049,456 targets, reproducing 3.017127224389041 exactly. Semantic tests cover initial equivalence, aggregate expert gradients, causal document isolation, sparse routing gradients, optimizer round trips and routing interventions. The notebook review executes locally; no complete Colab training run or successful remote push is claimed.

Source of results: original experiment JSON and logs under evidence/. Lesson source: supplied Session 14 PDF, pp. 36-39 and 52. Further technical references and counterexamples appear in study/CHALLENGE_THE_LESSON.md. All figures are from the measured experiments.